Laborator
Programimi i orientuar nga objektet (OOP)

Alion Emini

kurseinformatike.com

Përmbajtja e laboratorit

1. Qëllimi
2. Klasa `CellPhone`
3. Objekte në listë
4. Pickle dhe JSON
5. Klasa `Contact`
6. Trashëgimia
7. Mbyllje

1 Qëllimi

2 Klasa CellPhone

3 Objekte në listë

4 Pickle dhe JSON

5 Klasa Contact

6 Trashëgimia

7 Mbyllje

Çfarë do të bëjmë sot

- Të përforcojmë **programimin e orientuar drejt objekteve (OOP)** në Python.
- Të krijojmë klasën `CellPhone` me:
  - atributet private (`__manufact`, `__model`, `__retail_price`),
  - `__init__`, **setters** dhe **getters**.
- Të ruajmë objekte në:
  - **lista** (list of objects),
  - **fjalorë** (dictionary of objects).
- Të përdorim `pickle` për të ruajtur objekte në disk dhe për ti rikthyer.
- Të ndërtojmë një **mini menaxher kontaktesh** me klasën `Contact`.
- Të përdorim trashëgiminë për të shpalosur fuqinë e OOP.

1 Qëllimi

2 Klasa CellPhone
- Implementimi i klasës CellPhone
- Testimi i klasës CellPhone

3 Objekte në listë

4 Pickle dhe JSON

5 Klasa Contact

6 Trashëgimia

7 Mbyllje

1 Qëllimi

2 Klasa CellPhone
Implementimi i klasës CellPhone
Testimi i klasës CellPhone

3 Objekte në listë

4 Pickle dhe JSON

5 Klasa Contact

6 Trashëgimia

7 Mbyllje

Çfarë përfaqëson CellPhone

Qëllimi i ushtrimit

Të kuptosh si ndërtohet një klasë e thjeshtë Python dhe si përdoren **atributet private** dhe **metodat aksesuese**.

- Kompania **Wireless Solutions, Inc.** shet telefona.
- Na duhet një klasë që përfaqëson **një telefon të vetëm** në inventar.

Çfarë përfaqëson CellPhone

- Çdo objekt CellPhone do të ruajë:
  - **prodhuesin** (__manufact),
  - **modelin** (__model),
  - **çmimin me pakicë** (__retail_price).
- Klasa do të ketë:
  - __init__ për inicializim,
  - metoda set... për ndryshim,
  - metoda get... për lexim vlerash.

cellphone.py - përkufizimi i klasës I

```
class CellPhone:
    def __init__(self, manufact, model, price):
        self.__manufact = manufact        # prodhuesi
        self.__model = model              # modeli
        self.__retail_price = price       # cmimi me pakice

    def set_manufact(self, manufact):
        self.__manufact = manufact

    def set_model(self, model):
        self.__model = model
```

cellphone.py - përkufizimi i klasës II

```
    def set_retail_price(self, price):
        self.__retail_price = price

    def get_manufact(self):
        return self.__manufact

    def get_model(self):
        return self.__model

    def get_retail_price(self):
        return self.__retail_price
```

Vërejtje

Atributet fillojnë me __ për të treguar se janë **private** (d.m.th. nuk duam ti prekim direkt nga jashtë klasës).

1 Qëllimi

2 Klasa CellPhone
- Implementimi i klasës CellPhone
- Testimi i klasës CellPhone

3 Objekte në listë

4 Pickle dhe JSON

5 Klasa Contact

6 Trashëgimia

7 Mbyllje

Ideja e programit testues I

Qëllimi

Të shohim **në praktikë** se si:

- thirret konstruktori `__init__`,
- ruhet gjendja e objektit,
- lexohen vlerat me `get`....

Ideja e programit testues II

- Ti kërkojmë përdoruesit:
  - prodhuesin,
  - modelin,
  - çmimin me pakicë.
- Të krijojmë një objekt `CellPhone` me këto të dhëna.
- Të shfaqim atributet përmes **metodave** `get`....

cell_phone_test.py - program testues

```python
import cellphone

def main():
    man = input('Jep prodhuesin: ')
    mod = input('Jep numrin e modelit: ')
    retail = float(input('Jep cmimin me pakice: '))

    phone = cellphone.CellPhone(man, mod, retail)

    print('\nTe dhenat qe futet:')
    print(f'Prodhuesi: {phone.get_manufact()}')
    print(f'Modeli: {phone.get_model()}')
    print(f'Cmimi me pakice: {phone.get_retail_price():.2f} ')

if __name__ == '__main__':
    main()
```

Çfarë të provosh vetë

- Ekzekuto programin dhe fut disa vlera të ndryshme.
- Pas krijimit të objektit, provo:

```
phone.set_retail_price(149.99)
print(phone.get_retail_price())
```

- Shpjego me fjalët e tua:
  - Çfarë ndryshoi?
  - Pse është e dobishme të kemi metoda `set...`?

1 Qëllimi

2 Klasa CellPhone

3 Objekte në listë
- Versioni 1
- Versioni 2

4 Pickle dhe JSON

5 Klasa Contact

6 Trashëgimia

7 Mbyllje

1 Qëllimi

2 Klasa CellPhone

3 Objekte në listë
- Versioni 1
- Versioni 2

4 Pickle dhe JSON

5 Klasa Contact

6 Trashëgimia

7 Mbyllje

Lista me objekte CellPhone I

- Shpesh na duhen **shumë objekte** të së njëjtës klasë.
- Shembull: inventari i telefonave në magazinë.
- Zgjidhje natyrale: një **listë** me objekte `CellPhone`.
- Struktura:
  - funksioni `make_list()` mbledh të dhënat dhe krijon listën,
  - funksioni `display_list()` shfaq të dhënat e çdo objekti.

Lista me objekte CellPhone II

Ide algoritmike

Për çdo telefon:

1. lexo të dhënat,
2. krijo `CellPhone`,
3. `append` objektin në listë.

cell_phone_list.py - krijimi i listës I

```
import cellphone

def main():
    phones = make_list()
    print('Ketu jane te dhenat qe futet:\n')
    display_list(phones)
```

cell_phone_list.py - krijimi i listës II

```
def make_list():
    phone_list = []

    print('Jep te dhenat per pese telefona.\n')
    for count in range(1, 6):
        print(f'Telefoni nr. {count}:')
        man = input('  Prodhuesi: ')
        mod = input('  Modeli: ')
        retail = float(input('  Cmimi me pakice: '))
        print()

        phone = cellphone.CellPhone(man, mod, retail)
        phone_list.append(phone)

    return phone_list
```

cell_phone_list.py - krijimi i listës III

```
def display_list(phone_list):
    for item in phone_list:
        print('-----------------------')
        print(f'Prodhuesi: {item.get_manufact()}')
        print(f'Modeli   : {item.get_model()}')
        print(f'Cmimi    : {item.get_retail_price():.2f} ')
    print('-----------------------')

if __name__ == '__main__':
    main()
```

1 Qëllimi

2 Klasa CellPhone

3 Objekte në listë
- Versioni 1
- Versioni 2

4 Pickle dhe JSON

5 Klasa Contact

6 Trashëgimia

7 Mbyllje

Të modifikojmë...

Detyrë

Ndrysho programin `cell_phone_list.py` që:

- të pyesë përdoruesin **sa telefona** dëshiron të fusë (jo gjithmonë 5);
- të gjejë dhe shfaqë **telefonin më të shtrenjtë** në fund.

- Zgjidhja jonë:
  - ndryshojmë funksionin `make_list()` që të pyesë për numrin e telefonave;
  - shtojmë funksionin `find_most_expensive()` që përdor `max(...)` mbi listën;
  - në `main()` thërrasim këtë funksion dhe shfaqim telefonin më të shtrenjtë.

cell_phone_list2.py

```
import cellphone
def main():
    phones = make_list()

    if not phones:
        print('Nuk u fut asnje telefon.')
        return

    print('Ketu jane te dhenat qe futet:\n')
    display_list(phones)

    expensive = find_most_expensive(phones)

    print('\nTelefoni me i shtrenjte:')
    print('-----------------------')
    print(f'Prodhuesi: {expensive.get_manufact()}')
    print(f'Modeli   : {expensive.get_model()}')
    print(f'Cmimi    : {expensive.get_retail_price():.2f} ')
```

cell_phone_list2.py II

```python
def make_list():
    phone_list = []
    n = int(input('Sa telefona deshiron te fusesh? '))

    print()
    for count in range(1, n + 1):
        print(f'Telefoni nr. {count}:')
        man = input('  Prodhuesi: ')
        mod = input('  Modeli: ')
        retail = float(input('  Cmimi me pakice: '))
        print()

        phone = cellphone.CellPhone(man, mod, retail)
        phone_list.append(phone)

    return phone_list
```

cell_phone_list2.py III

```
def display_list(phone_list):
    for item in phone_list:
        print(f'Prodhuesi: {item.get_manufact()}')
        print(f'Modeli   : {item.get_model()}')
        print(f'Cmimi    : {item.get_retail_price():.2f} ')
        print()
```

cell_phone_list2.py IV

```
def find_most_expensive(phone_list):
    if not phone_list:
        return None
    return max(phone_list, key=lambda p: p.get_retail_price())

if __name__ == '__main__':
    main()
```

1 Qëllimi

2 Klasa CellPhone

3 Objekte në listë

4 Pickle dhe JSON
- picke dhe përdorimi i saj
- JSON dhe përdorimi i tij

5 Klasa Contact

6 Trashëgimia

7 Mbyllje

1 Qëllimi

2 Klasa CellPhone

3 Objekte në listë

4 Pickle dhe JSON
- picke dhe përdorimi i saj
- JSON dhe përdorimi i tij

5 Klasa Contact

6 Trashëgimia

7 Mbyllje

Pse të përdorim `pickle` me objekte?

- Duam të ruajmë inventarin e telefonave **midis ekzekutimeve**.
- Moduli `pickle` lejon:
  - **serializimin** (shndërrimin në byte stream),
  - ruajtjen në skedar binar (`.dat`),
  - **deserializimin** dhe rikthimin e objekteve.
- Mund të picklojmë:
  - fjalorë,
  - lista,
  - objekte të klasave tona (`CellPhone`, `Contact`, etj.).

Kujdes

Mos përdor `pickle.load` me skedarë nga burime të panjohura - mund të ekzekutojnë kod të dëmshëm.

pickle_cellphone.py - ruajtja e objekteve I

```
import pickle
import cellphone

FILENAME = 'cellphones.dat'

def main():
    again = 'y'

    output_file = open(FILENAME, 'wb')

    while again.lower() == 'y':
        man = input('Prodhuesi: ')
        mod = input('Modeli: ')
        retail = float(input('Cmimi me pakice: '))
        phone = cellphone.CellPhone(man, mod, retail)
        pickle.dump(phone, output_file)

        print('Te dhenat u ruajten.\n')
        again = input('Fut me shume telefona? (y/n): ')
```

pickle_cellphone.py - ruajtja e objekteve II

```
20
21      output_file.close()
22      print(f'Te dhenat u ruajten ne {FILENAME}.')
23
24  if __name__ == '__main__':
25      main()
```

unpickle_cellphone.py - lexim nga skedari I

```
import pickle
import cellphone

FILENAME = 'cellphones.dat'

def main():
    end_of_file = False

    input_file = open(FILENAME, 'rb')

    while not end_of_file:
        try:
            phone = pickle.load(input_file)
            display_data(phone)
        except EOFError:
            end_of_file = True

    input_file.close()
```

unpickle_cellphone.py - lexim nga skedari II

```
    def display_data(phone):
    print('----------------------')
    print(f'Prodhuesi: {phone.get_manufact()}')
    print(f'Modeli    : {phone.get_model()}')
    print(f'Cmimi     : {phone.get_retail_price():.2f} ')

if __name__ == '__main__':
    main()
```

1 Qëllimi

2 Klasa CellPhone

3 Objekte në listë

4 Pickle dhe JSON
- picke dhe përdorimi i saj
- JSON dhe përdorimi i tij

5 Klasa Contact

6 Trashëgimia

7 Mbyllje

Pse të përdorim JSON?

- **JSON** (JavaScript Object Notation) është një format i thjeshtë dhe i lexueshëm për ruajtjen dhe shkëmbimin e të dhënave.
- Përparësitë:
  - JSON është më i lehtë për t'u lexuar dhe shkruar krahasuar me `pickle`,
  - është i përdorshëm në shumë gjuhë programimi dhe platforma.
- Moduli `json` në Python ofron funksionalitete për:
  - **serializimin** dhe **deserializimin** e të dhënave në formate të lexueshme.

json_cellphone.py - ruajtja e objekteve në JSON I

```
import json
import cellphone

FILENAME = 'cellphones.json'

def main():
    again = 'y'
    phone_list = []

    while again.lower() == 'y':
        man = input('Prodhuesi: ')
        mod = input('Modeli: ')
        retail = float(input('Cmimi me pakice: '))
        phone = cellphone.CellPhone(man, mod, retail)
        phone_list.append({
            'manufact': phone.get_manufact(),
            'model': phone.get_model(),
            'retail_price': phone.get_retail_price()
        })
```

json_cellphone.py - ruajtja e objekteve në JSON II

```
20
21          print('Te dhenat u ruajten.\n')
22          again = input('Fut me shume telefona? (y/n): ')
23
24      with open(FILENAME, 'w') as output_file:
25          json.dump(phone_list, output_file, indent=4)
26
27      print(f'Te dhenat u ruajten ne {FILENAME}.')
28
29  if __name__ == '__main__':
30      main()
```

unjson_cellphone.py - lexim nga skedari JSON I

```python
import json

FILENAME = 'cellphones.json'

def main():
    with open(FILENAME, 'r') as input_file:
        phone_list = json.load(input_file)

    for phone in phone_list:
        print('----------------------')
        print(f'Prodhuesi: {phone["manufact"]}')
        print(f'Modeli   : {phone["model"]}')
        print(f'Cmimi    : {phone["retail_price"]:.2f} ')

if __name__ == '__main__':
    main()
```

JSON dhe Pickle - Diferencat kryesore

- **JSON** është një format tekstual që është i lehtë për t'u lexuar nga njerëzit dhe për përdorim ndërplatformë.
- **Pickle** është më efikas për ruajtjen e objekteve të Python, por është më i mbyllur dhe nuk mund të përdoret ndër platforma të ndryshme pa python.
- JSON është më i përshtatshëm për shkëmbim të të dhënave ndërmjet sistemeve të ndryshme (p.sh., mes aplikacioneve web dhe backend-it).

1 Qëllimi

2 Klasa CellPhone

3 Objekte në listë

4 Pickle dhe JSON

5 Klasa Contact
- Implementimi i klasës Contact
- Menaxheri i kontakteve me fjalor & pickle

6 Trashëgimia

7 Mbyllje

1 Qëllimi

2 Klasa CellPhone

3 Objekte në listë

4 Pickle dhe JSON

5 Klasa Contact
- Implementimi i klasës Contact
- Menaxheri i kontakteve me fjalor & pickle

6 Trashëgimia

7 Mbyllje

Nga telefona tek kontakte

- Tani duam të menaxhojmë **kontakte**:
  - emër,
  - numër telefoni,
  - email.
- Krijojmë klasën `Contact` me:
  - atributet private `__name`, `__phone`, `__email`,
  - metoda `set...` dhe `get...`,
  - metodën `__str__` për afishim të bukur.
- Kontaktesh do ti ruajmë në një **fjalor**:
  - **key** = emri i personit,
  - **value** = objekti `Contact`.

contact.py - përkufizimi i klasës I

```
class Contact:
    def __init__(self, name, phone, email):
        self.__name = name
        self.__phone = phone
        self.__email = email

    def set_name(self, name):
        self.__name = name

    def set_phone(self, phone):
        self.__phone = phone

    def set_email(self, email):
        self.__email = email
```

contact.py - përkufizimi i klasës II

```
    def get_name(self):
        return self.__name

    def get_phone(self):
        return self.__phone

    def get_email(self):
        return self.__email

    def __str__(self):
        return (f'Name : {self.__name}\n'
                f'Phone: {self.__phone}\n'
                f'Email: {self.__email}')
```

1 Qëllimi

2 Klasa CellPhone

3 Objekte në listë

4 Pickle dhe JSON

5 Klasa Contact

Implementimi i klasës Contact

Menaxheri i kontakteve me fjalor & pickle

6 Trashëgimia

7 Mbyllje

Ideja e programit contact_manager.py

- Ruajmë kontaktet në një **fjalor**:
  - `mycontacts[name] = Contact(...)`.
- Ofron menu:
  1. Kërko një kontakt
  2. Shto një kontakt
  3. Ndrysho një kontakt
  4. Fshi një kontakt
  5. Dil nga programi
- Përpara se të mbyllet, programi:
  - **picklon** fjalorin dhe e ruan në `contacts.dat`,
  - kur nis, mundohet ta **ngarkojë** fjalorin nga i njëjti skedar.

contact_manager.py - pjesa kryesore

```
import contact ; import pickle
LOOK_UP = 1; ADD = 2; CHANGE = 3; DELETE = 4; QUIT = 5
FILENAME = 'contacts.dat'

def main():
    mycontacts = load_contacts()

    choice = 0
    while choice != QUIT:
        choice = get_menu_choice()
        if choice == LOOK_UP: look_up(mycontacts)
        elif choice == ADD: add(mycontacts)
        elif choice == CHANGE: change(mycontacts)
        elif choice == DELETE: delete(mycontacts)

    save_contacts(mycontacts)

if __name__ == '__main__':
    main()
```

load_contacts & menuja I

```
def load_contacts():
    try:
        input_file = open(FILENAME, 'rb')
        contact_dct = pickle.load(input_file)
        input_file.close()
    except IOError:
        # Skedari nuk ekziston -> fjalor bosh
        contact_dct = {}
    return contact_dct
```

load_contacts & menuja II

```
def get_menu_choice():
    print('\nMenu')
    print('---------------------------')
    print('1. Kërko kontakt')
    print('2. Shto kontakt të ri')
    print('3. Ndrysho kontakt ekzistues')
    print('4. Fshi kontakt')
    print('5. Dil nga programi\n')

    choice = int(input('Zgjedhja jote: '))
    while choice < LOOK_UP or choice > QUIT:
        choice = int(input('Jep nje zgjedhje te vlefshme: '))
    return choice
```

look_up, add, change, delete I

```
def look_up(mycontacts):
    name = input('Jep emrin qe kerkon: ')
    print(mycontacts.get(name, 'Ky emer nuk u gjet.'))
```

look_up, add, change, delete II

```
def add(mycontacts):
    name = input('Emri: ')
    phone = input('Telefoni: ')
    email = input('Email: ')

    entry = contact.Contact(name, phone, email)

    if name not in mycontacts:
        mycontacts[name] = entry
        print('Kontakti u shtua.')
    else:
        print('Ky emer ekziston tashme.')
```

look_up, add, change, delete III

```
def change(mycontacts):
    name = input('Emri i kontaktit per ndryshim: ')
    if name in mycontacts:
        phone = input('Telefon i ri: ')
        email = input('Email i ri: ')
        entry = contact.Contact(name, phone, email)
        mycontacts[name] = entry
        print('Te dhenat u perditesuan.')
    else:
        print('Ky emer nuk u gjet.')
```

look_up, add, change, delete IV

```
def delete(mycontacts):
    name = input('Emri i kontaktit per fshirje: ')
    if name in mycontacts:
        del mycontacts[name]
        print('Kontakti u fshi.')
    else:
        print('Ky emer nuk u gjet.')
```

save_contacts - ruajtja në skedar

```
def save_contacts(mycontacts):
    output_file = open(FILENAME, 'wb')
    pickle.dump(mycontacts, output_file)
    output_file.close()
```

Pyetje për reflektim

- Pse është e përshtatshme të përdorim **fjalor** këtu (dhe jo vetëm listë)?
- Si na ndihmon `Contact.__str__` kur thërrasim `print(mycontacts[name])`?

1 Qëllimi

2 Klasa CellPhone

3 Objekte në listë

4 Pickle dhe JSON

5 Klasa Contact

6 Trashëgimia

7 Mbyllje

Të përdorim trashëgimin...

Skenari

Bank Financial Systems, Inc. zhvillon software financiar për banka dhe kreditori kursimi. Kompania po zhvillon një sistem të ri OOP për menaxhimin e llogarive të klientëve.

- Duam të modelojmë:
  - një **llogari kursimi** (`SavingsAccount`),
  - një **llogari Certificate of Deposit** (`CD`).
- Të dyja ruajnë:
  - numrin e llogarisë,
  - normën e interesit,
  - bilancin.
- `CD` ka edhe: **daten e maturimit**.

Analiza: ku futet trashëgimia?

- Vërejtje kyçe: `CD` është një **lloj i veçantë** i `SavingsAccount`.
- Pra:
  - `SavingsAccount` = klasa **bazë** (superclass),
  - `CD` = **nënklasë** (subclass) që shton `__maturity_date`.
- Plani:
  1. Të krijojmë modul `accounts.py` me klasat `SavingsAccount` dhe `CD`.
  2. Të krijojmë programin `account_demo.py` që i teston.

accounts.py - klasa SavingsAccount I

```
class SavingsAccount:
    def __init__(self, account_num, int_rate, bal):
        self.__account_num = account_num
        self.__interest_rate = int_rate
        self.__balance = bal

    def set_account_num(self, account_num):
        self.__account_num = account_num

    def set_interest_rate(self, int_rate):
        self.__interest_rate = int_rate

    def set_balance(self, bal):
        self.__balance = bal
```

accounts.py - klasa SavingsAccount II

```
    def get_account_num(self):
        return self.__account_num

    def get_interest_rate(self):
        return self.__interest_rate

    def get_balance(self):
        return self.__balance
```

Vërejtje

Atributet `__account_num`, `__interest_rate`, `__balance` janë **private** dhe kontrollohen përmes **setters/getters**.

accounts.py - klasa CD (subclass) I

```
class CD(SavingsAccount):
    def __init__(self, account_num, int_rate, bal, mat_date):
        SavingsAccount.__init__(self, account_num, int_rate, bal)
        self.__maturity_date = mat_date

    def set_maturity_date(self, mat_date):
        self.__maturity_date = mat_date

    def get_maturity_date(self):
        return self.__maturity_date
```

Çfarë po trashëgon CD?

- Të gjitha metodat e `SavingsAccount` (`get_balance`, `set_balance`, etj.).
- Të gjitha atributet private të inicializuara nga `SavingsAccount.__init__`.

account_demo.py - program testues I

```python
import accounts

def main():
    print('Jep te dhenat per nje llogari kursimi.')
    acct_num = input('Account number: ')
    int_rate = float(input('Interest rate: '))
    balance = float(input('Balance: '))

    savings = accounts.SavingsAccount(acct_num, int_rate, balance)
```

account_demo.py - program testues II

```
    print('\nJep te dhenat per nje CD.')
    acct_num = input('Account number: ')
    int_rate = float(input('Interest rate: '))
    balance = float(input('Balance: '))
    maturity = input('Maturity date (p.sh. 12/12/2024): ')

    cd = accounts.CD(acct_num, int_rate, balance, maturity)

    print('\nHere is the data you entered:\n')

    print('Savings Account')
    print('----------------')
    print(f'Account number: {savings.get_account_num()}')
    print(f'Interest rate: {savings.get_interest_rate()}')
    print(f'Balance: ${savings.get_balance():,.2f}')

    print('\nCD')
    print('----------------')
```

account_demo.py - program testues III

```
    print(f'Account number: {cd.get_account_num()}')
    print(f'Interest rate: {cd.get_interest_rate()}')
    print(f'Balance: ${cd.get_balance():,.2f}')
    print(f'Maturity date: {cd.get_maturity_date()}')

if __name__ == '__main__':
    main()
```

1 Qëllimi

2 Klasa CellPhone

3 Objekte në listë

4 Pickle dhe JSON

5 Klasa Contact

6 Trashëgimia

7 Mbyllje

Çfarë mësuam në këtë laborator

- Si të ndërtojmë klasa të thjeshta në Python (`CellPhone`, `Contact`).
- Si të përdorim **atribute private**, `__init__`, **setters** dhe **getters**.
- Si të ruajmë objekte në **lista** dhe **fjalorë**.
- Si të ruajmë/lexojmë objekte nga skedari me `pickle`.
- Si të ndërtojmë një **mini-aplikacion** si `contact_manager.py`.

Detyrë shtesë (opsionale)

Shtoni një **menu eksporti në JSON** për kontaktet, duke përdorur modulin `json`, në stil të ngjashëm me laboratorin e fjalorëve.

Suksese në laborator!

Pyetje?